

Aula 04 – Persistência Real com MySQL e SQLAlchemy

Objetivo da Aula: Compreender o conceito de ORM, conectar a aplicação FastAPI a um banco de dados MySQL e gerar automaticamente a tabela do inventário.

1. Preparando o Ambiente (MySQL e Dependências)

Antes do código, precisamos da infraestrutura.

- O Banco de Dados Local: Certifique-se de que os alunos estão com o serviço do MySQL rodando (geralmente via XAMPP, WAMP ou Docker no laboratório). Peça para eles abrirem o phpMyAdmin (ou DBeaver) e criarem um banco de dados vazio chamado `inova_inventario`.
- Novas Bibliotecas: No terminal, com o venv ativado, eles precisam instalar o SQLAlchemy (o ORM) e o PyMySQL (o driver que faz o Python conversar com o MySQL).

```
pip install sqlalchemy pymysql
```

```
(venv) C:\Users\          \Desktop\api-inventario-maker-3F-TurmaB>pip install sqlalchemy pymysql
Collecting sqlalchemy
  Downloading sqlalchemy-2.0.50-cp314-cp314-win_amd64.whl.metadata (9.8 kB)
Collecting pymysql
  Downloading pymysql-1.2.0-py3-none-any.whl.metadata (4.3 kB)
```

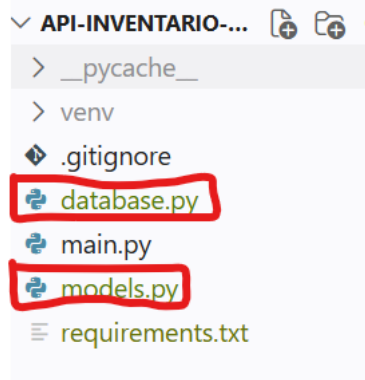
- Atualizando o Requirements:

```
pip freeze > requirements.txt
```

```
(venv) C:\Users\          \Desktop\api-inventario-maker-3F-TurmaB>pip freeze > requirements.txt
```

2. Organizando a Casa (Separando as Responsabilidades)

Neste momento não devemos colocar tudo no `main.py`. Vamos criar dois arquivos novos na raiz do projeto: `database.py` e `models.py`.



3. O Arquivo database.py

Este arquivo será o responsável por estabelecer e gerenciar a conexão com o MySQL.

Arquivo: database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import declarative_base
from sqlalchemy.orm import sessionmaker
```

String de conexão:

banco://usuario:senha@servidor:porta/nome_do_banco

(Ajuste o root e a senha vazia conforme o padrão das máquinas do laboratório)

```
URL_BANCO_DADOS = "mysql+pymysql://root:@localhost:3306/inova_inventario"
```

O Engine é o motor que traduz o Python para o dialeto do MySQL

```
engine = create_engine(URL_BANCO_DADOS)
```

A Sessão é o "túnel" por onde os dados vão trafegar

```
SessaoLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Base para criar as tabelas

```
Base = declarative_base()
```

Função auxiliar para injetar a sessão nas rotas (veremos na próxima aula)

```
def obter_banco():
```

```
    banco = SessaoLocal()
```

```
    try:
```

```
        yield banco
```

```
finally:  
    banco.close()
```

4. O Arquivo models.py (A Tabela Física)

É aqui que muitos alunos confundem. Perceba a diferença entre o Pydantic (A "catraca" que valida o JSON que chega da internet) e o SQLAlchemy (O "molde" que cria a tabela de ferro dentro do MySQL).

```
# Arquivo: models.py  
from sqlalchemy import Column, Integer, String  
from database import Base  
  
class Componente(Base):  
    # Nome exato da tabela no MySQL  
    __tablename__ = "tb_componentes"  
  
    # Colunas da tabela  
    id = Column(Integer, primary_key=True, index=True, autoincrement=True)  
    nome = Column(String(100), nullable=False)  
    quantidade = Column(Integer, default=0)  
    categoria = Column(String(50), nullable=False)
```

5. O Script de Criação Automática (No main.py)

Para fechar a aula com uma vitória visual, vamos fazer o Python se conectar ao MySQL e criar a tabela tb_componentes, sem precisarmos escrever nenhuma linha de SQL (CREATE TABLE...).

Voltar ao main.py e adicionar estas duas linhas logo abaixo das importações, antes de instanciar o app = FastAPI(...):

```
# No topo do arquivo main.py  
import models  
from database import engine  
  
# Cria as tabelas no banco de dados automaticamente ao iniciar  
models.Base.metadata.create_all(bind=engine)
```

- Abra o PhpMyAdmin e crie o banco de dados chamado: `inova_inventario`.
- **A Grande Validação Final:** Subir o servidor novamente com `uvicorn main:app --reload`. Em seguida, ir ao phpMyAdmin e atualizar a tela. Veja que a tabela `tb_componentes` foi criada automaticamente com todas as colunas configuradas perfeitamente!

The screenshot shows the phpMyAdmin interface. On the left, the database 'inova_inventario' is selected, and the table 'tb_componentes' is highlighted with a red arrow. On the right, the 'Estrutura da tabela' (Table Structure) tab is active, showing the table's schema. The schema is enclosed in a red box and contains the following columns:

#	Nome	Tipo	Colaço	Atributos	Nulo
1	id	int(11)			Não
2	nome	varchar(100)	utf8mb4_general_ci		Não
3	quantidade	int(11)			Sim
4	categoria	varchar(50)	utf8mb4_general_ci		Não

Below the table structure, there are options to 'Marcar todos' (Mark all), 'Com marcados:' (With marked:), and 'Visualizar' (View). There are also buttons for 'Espacial' (Spatial), 'Texto completo' (Full text), and 'Adicionar coluna(s) central(is)' (Add column(s) central(is)).